

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ПРАКТИЧЕСКОЙ РАБОТЕ №2
дисциплины «Программирование на Python»

Выполнил:
Скоробогатов Виктор Андреевич
1 курс, группа ИВТ-б-о-24-1,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Руководитель практики:
Воронкин Роман Александрович,
доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2025 г.

Тема: Основы языка Python

Цель: исследование процесса установки и базовых возможностей языка Python версии 3.x.

Ход работы

[Ссылка на GitHub репозиторий](#)

Для начала работы был создан публичный репозиторий, выбрав язык программирования Python и лицензию MIT. Затем репозиторий был клонирован на рабочий компьютер. Файл .gitignore был дополнен для работы со средой разработки PyCharm. Также была организована структура ветвления проекта в соответствии с моделью git-flow:

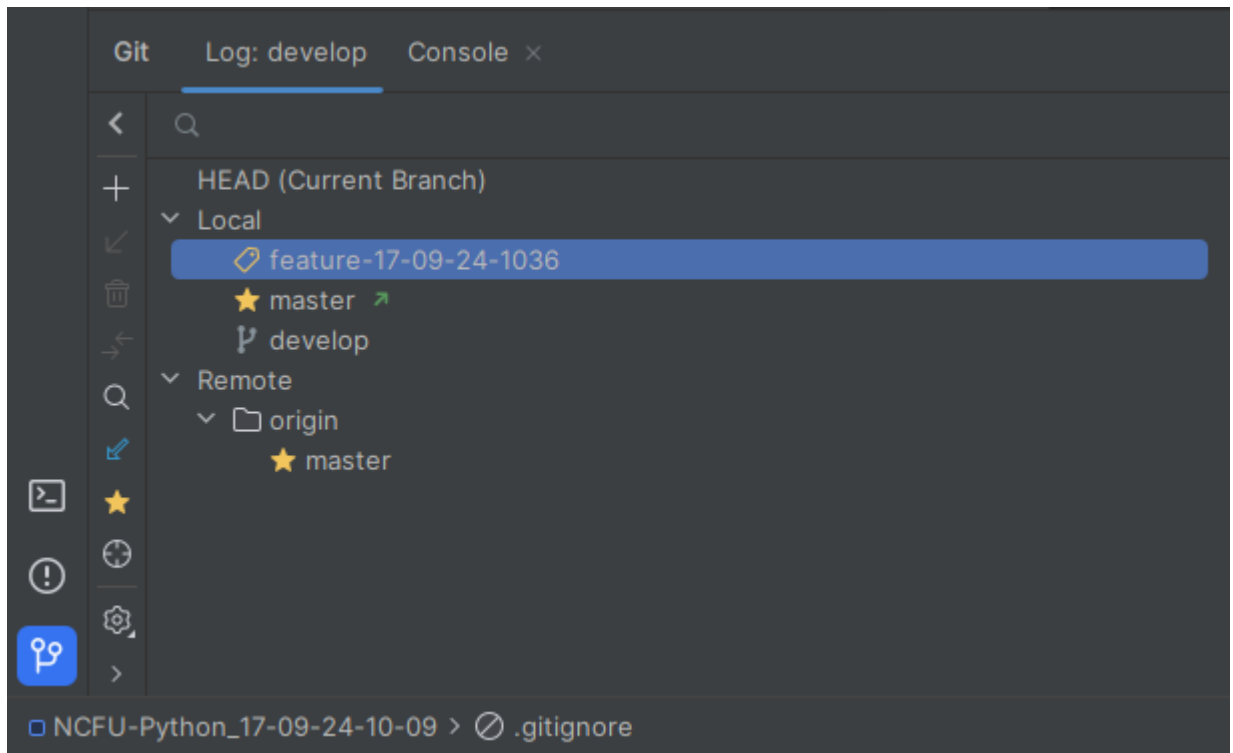


Рисунок 1 – Добавление ветки develop и feature

Далее был создан проект PyCharm в папке репозитория и начата работа над решением задач с помощью языка программирования Python3, создан файл user.py и написана программа, описанная в пункте 8 методических указаний:

```

D:\Учеба\PythonNCFU\NCFU-Python_17-09-24-10-09\.venv\Scripts\python.exe
Hello, what`s your name?
>Victor
How old are you?
>20
Where are you live?
>Stavropol
This is Victor
It is 20
(S)he live in Stavropol

```

Рисунок 2 – Результат работы программы user.py

Далее по порядку были выполнены другие задания по написанию программ:

```

D:\Учеба\PythonNCFU\NCFU-Python_17-09-24-10-09\.venv\Scripts\python.exe D
Solve: 4 * 100 - 54 = ?
>346
Your answer: 346
Right answer: 346
# Здесь преобразование типов необязательно из-за того, что мы
# в подследствии не используем наши данные в вычислениях,
# поэтому можно оставить их в строчном типе

```

Рисунок 3 – Результат работы программы arithmetic.py

```

numbers x
D:\Учеба\PythonNCFU\NCFU-Python_17-09-24-10-09\.venv\Scripts\python.exe
Enter 4 numbers:
>6
>0
>9
>0
0.67

```

Рисунок 4 – Результат работы программы numbers.py

Индивидуальное задание:

Даны основания и высота равнобедренной трапеции. Найти её периметр.

```

1  base_small = int(input("Enter first base (smaller):\n>"))
2  base_big = int(input("Enter second base (bigger):\n>"))
3  height = int(input("Enter height:\n>"))
4
5  base_right_angle_triangle = (base_big - base_small) / 2
6  hypotenuse = (height**2 + base_right_angle_triangle**2)
7
8  perimetr = (hypotenuse * 2) + base_big + base_small
9
10 print(f"Perimetr of trapezoid: {perimetr}")
11

```

Рисунок 5 – Код программы individual.py

```

D:\Учеба\PythonNCFU\NCFU-Python_17-09-24-10-09\.venv\Scripts\python.exe D:\Учеба\PythonNCFU\NCFU-Python_17-09-24-10-09\individual.py
Enter first base (smaller):
>12
Enter second base (bigger):
>27
Enter height:
>14
Perimetr of trapezoid: 543.5

```

Рисунок 6 – Результат работы программы individual.py

Задача повышенной сложности:

Даны два целых числа a и b . Если a делится на b или b делится на a , то вывести 1, иначе – любое другое число. Условные операторы и операторы цикла не использовать.

```

1  a = int(input("Enter number a:\n>"))
2  b = int(input("Enter number b:\n>"))
3
4  print(int(a % b == 0 or b % a == 0))

```

Рисунок 7 – Код программы hard_task.py

```

D:\Учеба\PythonNCFU\NCFU-Python_17-09-24-10-09\.venv\Scripts\python.exe D:\Учеба\PythonNCFU\NCFU-Python_17-09-24-10-09\hard_task.py
Enter number a:
>2
Enter number b:
>8
1

```

Рисунок 8 – Результат работы программы hard_task.py

```
$ git merge develop
Updating a85d06d..5b0feb0
Fast-forward
 .gitignore | 63 ++++++
 .idea/NCFU-Python_17-09-24-10-09.iml | 10 +++
 .idea/inspectionProfiles/profiles_settings.xml | 6 ++
 .idea/misc.xml | 4 ++
 .idea/modules.xml | 8 +++
 .idea/vcs.xml | 6 ++
 arithmetic.py | 5 ++
 hard_task.py | 4 ++
 individual.py | 10 +++
 numbers.py | 9 +++
 user.py | 7 +++
11 files changed, 105 insertions(+), 27 deletions(-)
create mode 100644 .idea/NCFU-Python_17-09-24-10-09.iml
create mode 100644 .idea/inspectionProfiles/profiles_settings.xml
create mode 100644 .idea/misc.xml
create mode 100644 .idea/modules.xml
create mode 100644 .idea/vcs.xml
create mode 100644 arithmetic.py
create mode 100644 hard_task.py
create mode 100644 individual.py
create mode 100644 numbers.py
create mode 100644 user.py
```

Рисунок 9 – Слияние веток master и develop

Ответы на контрольные вопросы:

1. Windows: скачать установщик с python.org, запустить, отметить "Add Python to PATH", установить. Linux: обычно предустановлен; иначе — через пакетный менеджер (`sudo apt install python3`).
2. Anaconda — дистрибутив Python с предустановленными библиотеками для Data Science и менеджером сред conda. Официальный Python — чистая версия без дополнительных пакетов.
3. В терминале выполнить: `conda --version` или `conda list`. Если команды работают — Anaconda установлена.
4. PyCharm: File → Settings → Project → Python Interpreter → выбрать нужный интерпретатор.
5. Нажать `Ctrl+Shift+F10` (текущий файл) или кнопку Run (зелёный треугольник).
6. Интерактивный — ввод команд в реальном времени (через python в консоли). Пакетный — запуск готового .py файла.
7. Динамическая типизация — тип переменной определяется в runtime и может меняться.
8. Основные типы: `int`, `float`, `str`, `bool`, `list`, `tuple`, `dict`, `set`, `NoneType`.
9. Объекты создаются при присваивании значения. Переменная — ссылка на объект в памяти. Присваивание связывает имя с объектом.
10. `import keyword; print(keyword.kwlist)`

11. `id()` — возвращает идентификатор объекта в памяти. `type()` — возвращает тип объекта.

12. Изменяемые (mutable): `list`, `dict`, `set` — можно менять без создания нового объекта. Неизменяемые (immutable): `int`, `str`, `tuple` — нельзя изменить.

13. `/` — обычное деление (возвращает `float`). `//` — целочисленное деление (возвращает `int`).

14. Встроенная поддержка: `z = 3 + 4j`. Модуль `cmath` для математических операций.

15. `math` — математические функции для вещественных чисел. `cmath` — аналогичные функции для комплексных чисел.

16. `sep` — разделитель между аргументами (по умолчанию пробел). `end` — что выводить в конце (по умолчанию `\n`).

17. `format()` — форматирование строк через `"{}".format(value)`. f-строки: `f"{variable}"` — более современный и удобный способ.

18. `int(input())` — для целого числа. `float(input())` — для вещественного.

Вывод: в ходе работы был исследован процесс установки и базовые возможности языка Python версии 3.x.